Temporal Difference: Expected SARSA

This notebook illustrates the **Expected SARSA** method, which is an on-policy TD control algorithm.

The test case

The environment is a 6x6 grid, where the agent starts at the top-left corner (state S_0) and aims to reach the bottom-right corner (state S_{35}). The agent can move in four directions: up, down, left, and right. However, there are obstacles (walls) at states S_{14}, S_{20}, S_{21} and S_{27} that the agent cannot pass through, and a pit at state S_{16} where the agent would get stuck. The objective is modeled by assigning a reward to each state: the pit has a reward of -10000, and the end goal a reward of 10. Passing through any other state has a reward of -1. The agent receives the reward immediately upon entering a state. The episode ends when the agent reaches either the end goal or the pit. Maximixing the reward is equivalent to finding the shortest feasible path to the goal.

Gridworld with obstacles and a pit

S0 (Start)	S1	S2	S3	S4	S5
S6	S7	S8	S9	S10	S11
S12	S13	S14 (Wall)	S15	S16 (Pit)	S17
S18	S19	S20 (Wall)	S21 (Wall)	S22	S23
S24	S25	S26	S27 (Wall)	S28	S29
S30	S31	S32	S33	S34	S35 (Goal)

The Temporal Difference prediction method.

Recall the update rule for $V_\pi(S_t)$:

$$V_\pi(S_t) \leftarrow V_\pi(S_t) + \alpha[G_t - V_\pi(S_t)]$$

where G_t is the return following a visit (first visit or every-visit) to S_t. This update can only be performed at the end of the episode, when G_t is known.

The principle of TD prediction is to update $V_\pi(S_t)$ at time $t+1$, by approximating G_t by $R_{t+1} + \gamma V(S_{t+1})$. The updating rule becomes:

$$V_\pi(S_t) \leftarrow V_\pi(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V_\pi(S_t)]$$

In this formulation, we compare $V(S_t)$ to an estimate computed at time $t+1$, hence the name "Temporal Difference".

We repeatedly use the following helper function to compute the next state s' and reward $R(s,a)$ for a given state and action.

```
ACTIONS = [(-1,0), (1,0), (0,-1), (0,1)]  # Up, Down, Left, Right

slippage = {
        0: [0, 2, 3], # Up -> Up, Left, Right
        1: [1, 2, 3], # Down -> Down, Left, Right
        2: [2, 0, 1], # Left -> Left, Up, Down
        3: [3, 0, 1]  # Right -> Right, Up, Down
}

# probabilities of move consistent with action and of slippage
probs = [0.8, 0.1, 0.1]

def move(s, a_idx):
    """
    Returns the next state and reward for taking action a_idx in state s.
    """
    row, col = divmod(s, GRID_SIZE)
    dr, dc = ACTIONS[a_idx]
    nr = max(0, min(GRID_SIZE - 1, row + dr))
    nc = max(0, min(GRID_SIZE - 1, col + dc))
    next_s = nr * GRID_SIZE + nc

    if next_s in OBSTACLES:
        # bounce off the obstacle
        next_s = s

    if next_s in GOAL_STATES:
        reward = GOAL_REWARD
    elif next_s in PIT_STATES:
        reward = PIT_REWARD
    else:
        reward = MOVE_REWARD

    return next_s, reward
```

Expected SARSA Algorithm

Expected SARSA is an **on-policy** algorithm that is often more stable than regular SARSA. Instead of updating based on a single action A_{t+1} chosen by the policy, it updates based on the **expected value** of the following state under the current policy.

The update rule is:

$$Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma \sum_a \pi(a|S')Q(S',a) - Q(S,A)]$$

This makes it less sensitive to the "noise" of exploratory moves while staying on-policy.

```
# --- Constants ---
ALPHA = 0.1
EPSILON_START = 0.5
EPSILON_END = 0.01
GAMMA = 0.99
EPISODES = 150000
EPISODES_2 = 100000
MAX_STEPS = 200

def get_epsilon(episode, total_episodes):
    # Steady decay for discovery
    if episode < EPISODES_2:
        return 1.0 - (episode / EPISODES_2) * 0.99
    else:
        return 0.01

# --- Environment Logic ---
def step(s, a_idx):
    """
    Performs the action a_idx in state s and returns the next state and reward. Slippage occu
    """
    # 80% success, 10% slippage on each side
    r = random.random()
    if r < probs[0]:
        actual_a = slippage[a_idx][0]
    elif r < (probs[0]+probs[1]):
        actual_a = slippage[a_idx][1]
    else:
        actual_a = slippage[a_idx][2]

    next_s, reward = move(s, actual_a)
    return next_s, reward


def choose_action(s, Q, epsilon):
    """
    Epsilon-greedy action selection.
```

```
    """
    if random.random() < epsilon:
        return random.randint(0, 3)
    else:
        return np.argmax(Q[s])

def set_reproducibility(seed):
    random.seed(seed)
    np.random.seed(seed)

# --- Expected SARSA Training Loop ---
Q = np.zeros((STATES, 4))
epsilon = EPSILON_START
# Use any integer
SEED = 42
set_reproducibility(SEED)

# Initialize a counter
state_visits = np.zeros(STATES)
import os
import csv

# Delete the file if it exists
csv_log_path = "expected_sarsa.csv"
if os.path.exists(csv_log_path):
    os.remove(csv_log_path)

for episode in range(EPISODES):
    s = 0 # Start state
    steps = 0
    eps = get_epsilon(episode, EPISODES)

    while s not in (GOAL_STATES + PIT_STATES) and steps <= MAX_STEPS:
        a = choose_action(s, Q, eps)
        next_s, reward = step(s, a)

        # Expected SARSA Update (On-policy)
        if next_s in (GOAL_STATES + PIT_STATES):
            td_target = reward
        else:
            # Calculate expectation over best action and random drift
            q_max = np.max(Q[next_s])
```

```
            q_mean = np.mean(Q[next_s])
            expected_v = (1 - eps) * q_max + eps * q_mean
            td_target = reward + GAMMA * expected_v

        td_error = td_target - Q[s, a]
        Q[s, a] += ALPHA * td_error
        s = next_s
        steps += 1
        state_visits[s] += 1

    if (episode < 200) or (episode % 500 == 0):
        csv_debug(csv_log_path, episode=episode,
        epsilon=eps,
        Q_Up=Q[15, 0],
        Q_Down=Q[15, 1],
        Q_LEFT=Q[15, 2],
        Q_RIGHT=Q[15, 3],
        visits=state_visits[15])


# --- Extract Optimal Policy ---
optimal_policy = np.argmax(Q, axis=1)

logging.shutdown()
```

The figure below shows the optimal policy found by Expected SARSA. By averaging over next actions, this method correctly identifies the safe routes even in high-risk zones.

Hyperparameter	Value	Remark
α	0.100	Stability for large penalties
γ	0.990	
ϵ_{start}	0.500	Full initial exploration
ϵ_{end}	0.010	Force deterministic safety at end
Episodes	150000	
Max Steps	200	about 20 times the shortest path
Pit Penalty	-10000	low risk tolerance

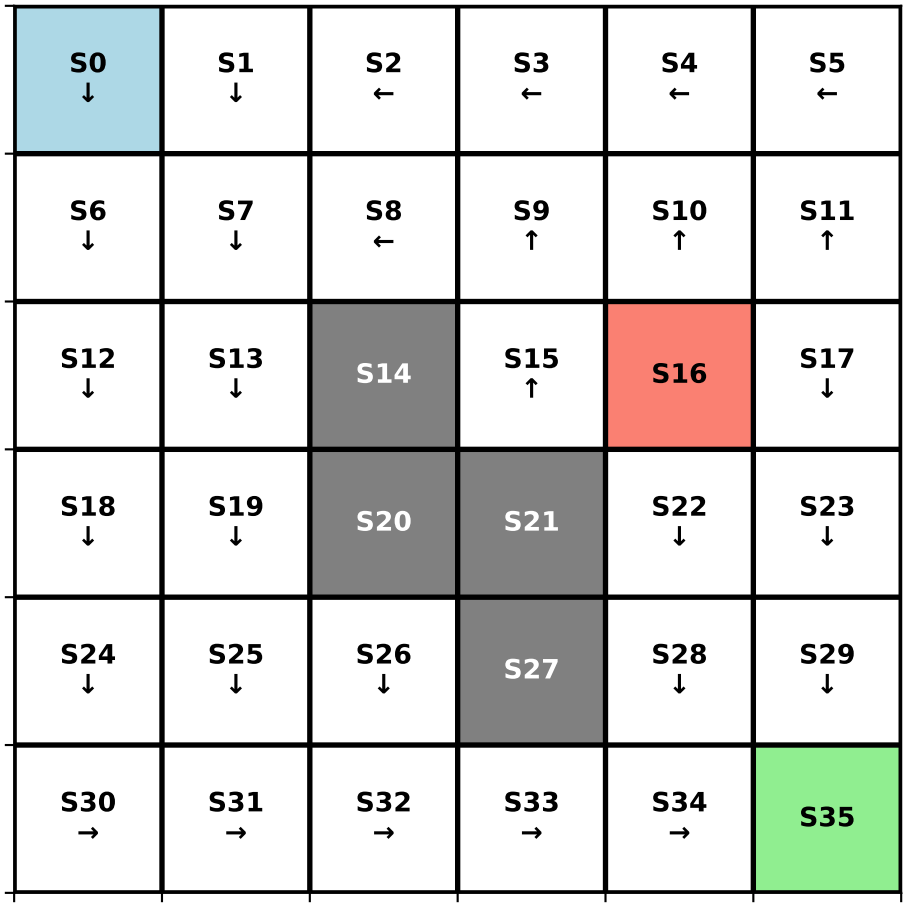

Figure 1: Optimal policy found by Expected SARSA